

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO LINUX

Linux is an open source operating system (OS). An operating system is the software that directly manages a system's hardware and resources, like CPU, memory, and storage. The OS sits between applications and hardware and makes the connections between all of your software and the physical resources that do the work.

Think about an OS like a car engine. An engine can run on its own, but it becomes a functional car when it's connected with a transmission, axles, and wheels. Without the engine running properly, the rest of the car won't work.

1.2 WORKING

Linux was designed to be similar to UNIX, but has evolved to run on a wide variety of hardware from phones to supercomputers. Every Linux-based OS involves the Linux kernel—which manages hardware resources—and a set of software packages that make up the rest of the operating system.

The OS includes some common core components, like the GNU tools, among others. These tools give the user a way to manage the resources provided by the kernel, install additional software, configure performance and security settings, and more.

1.3 LINUX ARCHITECTURE DESIGN

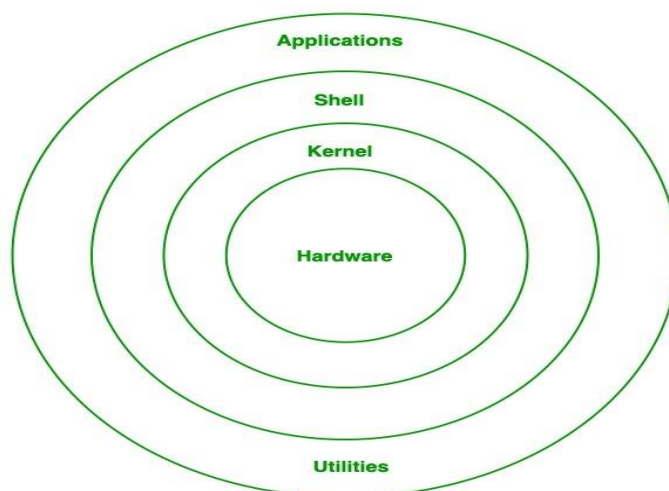


FIG 1.1: LINUX ARCHITECTURE DESIGN

Linux architecture has the following components:

1. **Kernel:** Kernel is the core of the Linux based operating system. It virtualizes the common hardware resources of the computer to provide each process with its virtual resources. This makes the process seem as if it is the sole process running on the machine. The kernel is also responsible for preventing and mitigating conflicts between different processes. Different types of the kernel are:
 - Monolithic Kernel
 - Hybrid kernels
 - Exo kernels
 - Micro kernels
2. **System Library:** Linux uses system libraries, also known as shared libraries, to implement various functionalities of the operating system. These libraries contain pre-written code that applications can use to perform specific tasks. By using these libraries, developers can save time and effort, as they don't need to write the same code repeatedly. System libraries act as an interface between applications and the kernel, providing a standardized and efficient way for applications to interact with the underlying system.
3. **Shell:** The shell is the user interface of the Linux Operating System. It allows users to interact with the system by entering commands, which the shell interprets and executes. The shell serves as a bridge between the user and the kernel, forwarding the user's requests to the kernel for processing. It provides a convenient way for users to perform various tasks, such as running programs, managing files, and configuring the system.
4. **Hardware Layer:** The hardware layer encompasses all the physical components of the computer, such as RAM (Random Access Memory), HDD (Hard Disk Drive), CPU (Central Processing Unit), and input/output devices. This layer is responsible for interacting with the Linux Operating System and providing the necessary resources for the system and applications to function properly.
5. **System Utility:** System utilities are essential tools and programs provided by the Linux Operating System to manage and configure various aspects of the system. These utilities perform tasks such as installing software, configuring network settings, monitoring system performance, managing users and permissions, and much more. System utilities simplify system administration tasks, making it easier for users to maintain their Linux systems efficiently.

CHAPTER 2

INSTALLATION

THERE ARE TWO WAYS OF INSTALLING LINUX: -

2.1 Normal Installation

2.2 Installation Through Virtual Box

1. **The first way** is to download the Linux distribution you want and burn it into a DVD or USB stick and boot your machine with it and complete the installation process.

2. **The Second way** is to install it virtually on a virtual machine like VirtualBox or VMware without touching your Windows or Mac system, so your Linux system will be contained in a window you can minimize and continue working on your real system.

We can install it in windows in the following ways:-

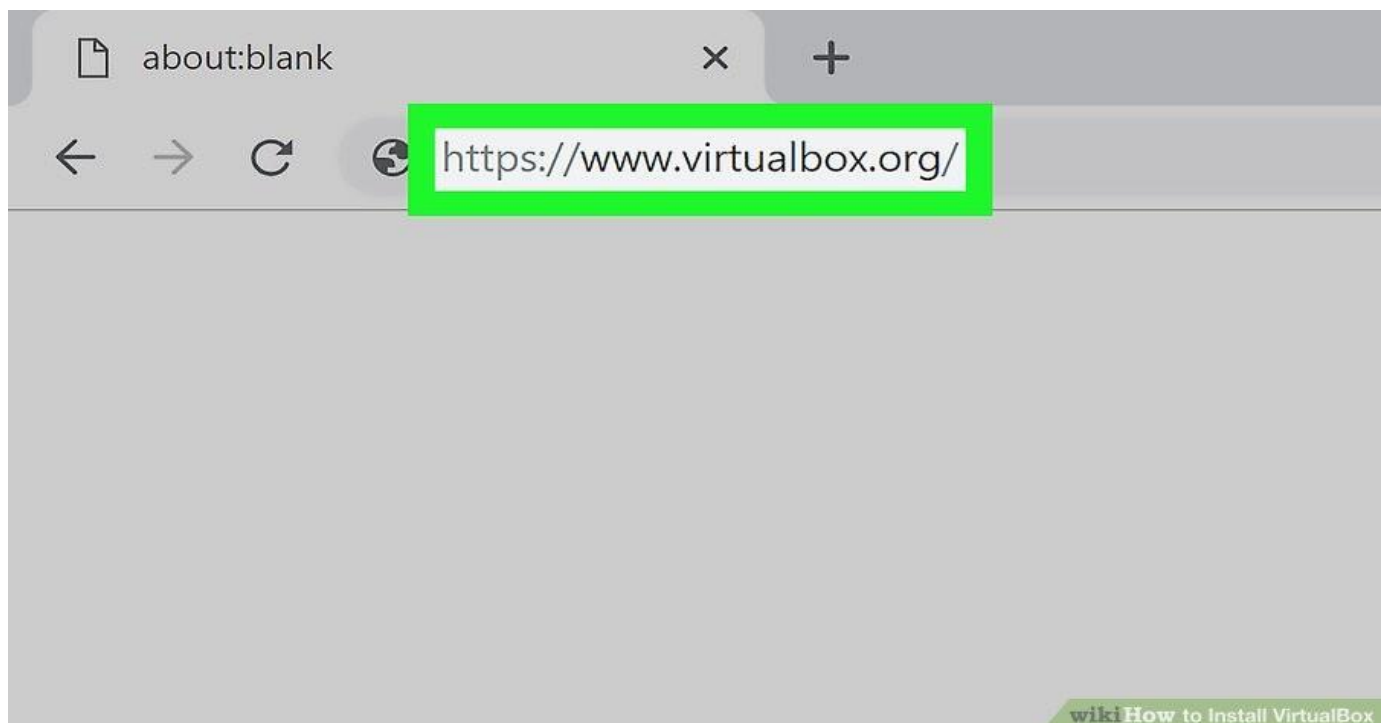


FIG 2.1: VIRTUAL BOX WEBSITE

Open the Virtual Box website.

Go to <https://www.VirtualBox.org/> in your computer's Internet browser. This is the website from which you'll download the VirtualBox setup file.



FIG 2.2: VIRTUAL BOX VERSION 5.2

Click Download VirtualBox.

It's a blue button in the middle of the page. Doing so will open the downloads page.

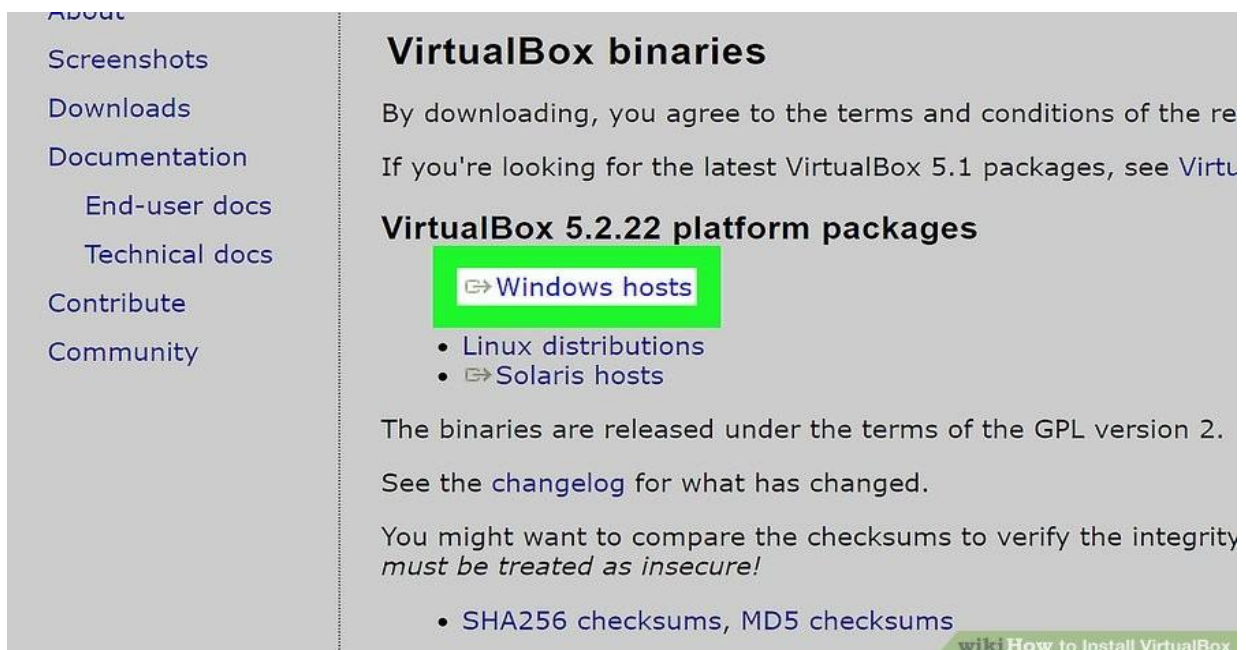


FIG 2.3: PLATFORM PACKAGES

Click Windows hosts.

You'll see this link below the "Virtual Box 5.2.8 platform packages" heading. The Virtual Box EXE file will begin downloading onto your computer.

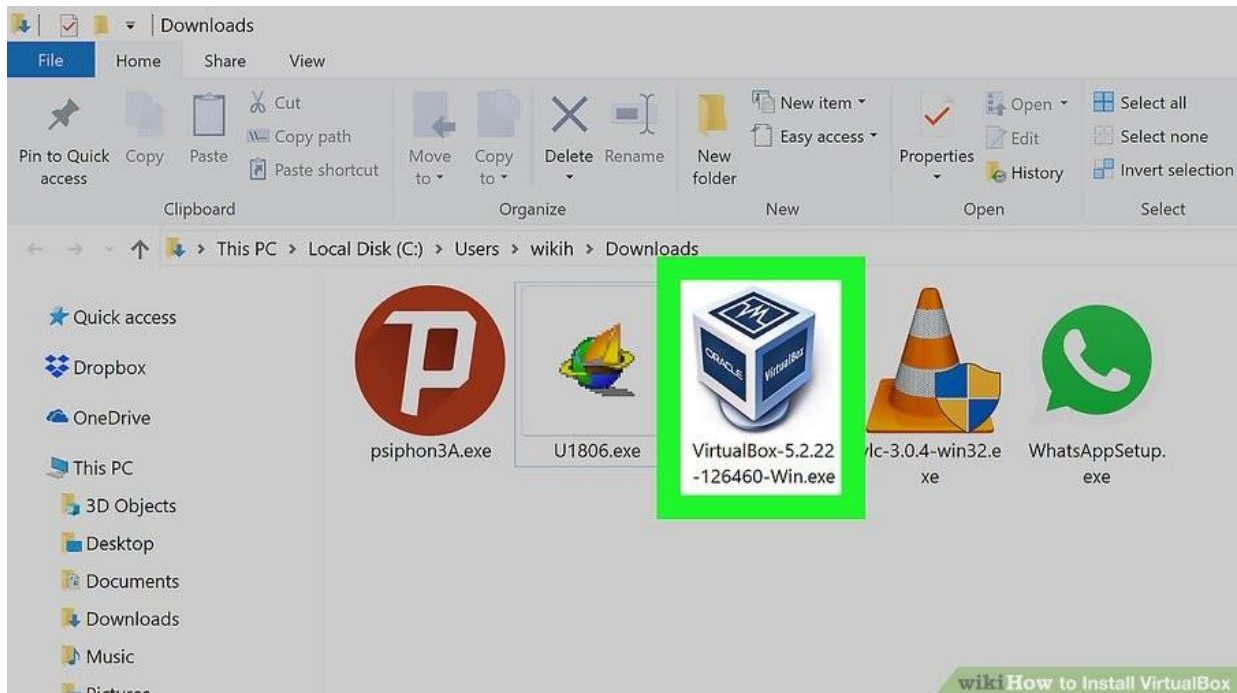


FIG 2.4: VIRTUAL BOX EXE FILE

Open the Virtual Box EXE file.

Go to the location to which the EXE file downloaded and double-click the file. Doing so will open the Virtual Box installation window.



FIG 2.5: ORACLE VM SETUP

CREATING A VIRTUAL MACHINE

- Gather your installation disc(s) or files.
- When creating a virtual machine, you will need to install the operating system just like you would on a regular computer. This means that you will need the installation disc(s) for the operating system you want to install on the virtual machine.
- You can also install an operating system by using its ISO file.

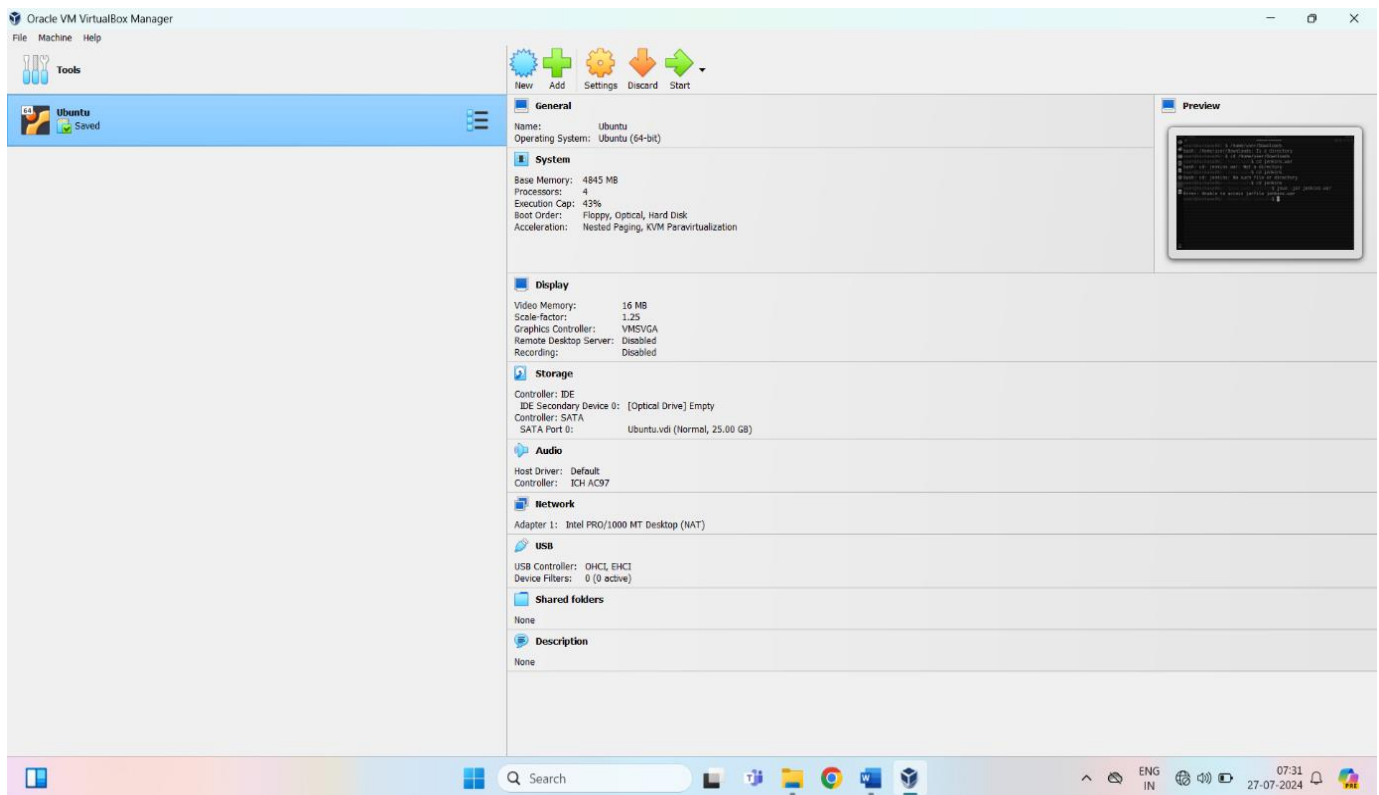


FIG 2.6: CREATING A VIRTUAL MACHINE

CHAPTER 3

BASIC LINUX COMMANDS

3.1 BASIC COMMANDS

What is command?

This command gives a one-line description about the command. It can be used as a quick reference for any command.

Manual Pages

‘—help’ option and ‘what is’ command do not provide through information about the command. For more detailed information, Linux provides man pages and info pages. To see a command’s manual page, man command is used.

1. pwd Command:

‘pwd’ command prints the absolute path to current working directory.

```
$ pwd
/home/Linux
```

2. cal command:

Displays the calendar of the current month.

```
$ cal
July 2024
Su Mo Tu We ThFr Sa
1 2 3 4 5 6 7
8 9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31
```

3. echo command:

This command will echo whatever you provide it.

```
$ echo "Linux intern"  
Linux intern
```

The 'echo' command is used to display the values of a variable. One such variable is 'HOME'. To check the value of a variable, precede the variable with a \$ sign.

4. date command:

Displays current time and date.

```
$ date  
Fri Jul 6 01:07:09 IST 2024
```

If you are interested only in time, you can use 'date+%T' (in hh:mm:ss);

5. tty command:

Displays current terminal.

```
$ tty  
/dev/pts/0
```

6. whoami command:

This command reveals the user who is currently logged in.

```
$ whoami  
Linux
```

7. id command:

This command prints user and groups (UID and GID) of the current user.


```
$ id
uid=1000(Linux) gid=1000(Linux)
groups=1000(Linux),4(adm),20(dialout),24(cdrom)
,46(plugdev),112(lpadmin),120(admin),122(sam-
bashare)
```

By default, information about the current user is displayed. If another username is provided as an argument, information about that user will be printed:

8. **clear command:**

This command clears the screen.

9. **help option:**

With almost every command, ‘-help’ option shows usage summary for that command.

\$date—help

```
Usage: date [OPTION]... [+FORMAT] or: date [-u|--utc|--univer-sal]
[MMDDhhmm[[CC]YY][.ss]]
```

10. **Display the current time:**

In the given FORMAT, or set the system date.

CHAPTER 4

FILE HANDLING

4.1 LISTING FILES

To list the files and directories stored in the current directory, use the following command –

```
$ls
```

Here is the sample output of the above command –

```
$ls
```

```
bin          hosts      lib        res.03
ch07         hw1       pub       test_results
ch07.bak     hw2       res.01    users
docs        hw3       res.02    work
```

The command `ls` support the `-l` option which would help you to get more information about the listed files

```
$ls -l
```

```
total 1962188
```

```
drwxrwxr-x   2 amroodamrood          4096 Dec 25 09:59 uml
-rw-rw-r--   1 amroodamrood          5341 Dec 25 08:38 uml.jpg
drwxr-xr-x   2 amroodamrood          4096 Feb 15   2006 univ
drwxr-xr-x   2 root                root      4096 Dec    9   2007 urls-
pedia
-rw-r--r--   1 root                root      276480 Dec    9   2007 urls-pedia.ta
drwxr-xr-x   8 root                root      4096 Nov 25   2007 usr
drwxr-xr-x   2      200      300      4096 Nov 25   2007
webthumb-1.01
-rwxr-xr-x   1 root                root      3192 Nov 25   2007
webthumb.php
-rw-rw-r--   1 amroodamrood          20480 Nov 25   2007
webthumb.tar
```

Here is the information about all the listed columns –

First Column – Represents the file type and the permission given on the file. Below is the description of all type of files.

Second Column – Represents the number of memory blocks taken by the file or directory.

Third Column – Represents the owner of the file. This is the Unix user who created this file.

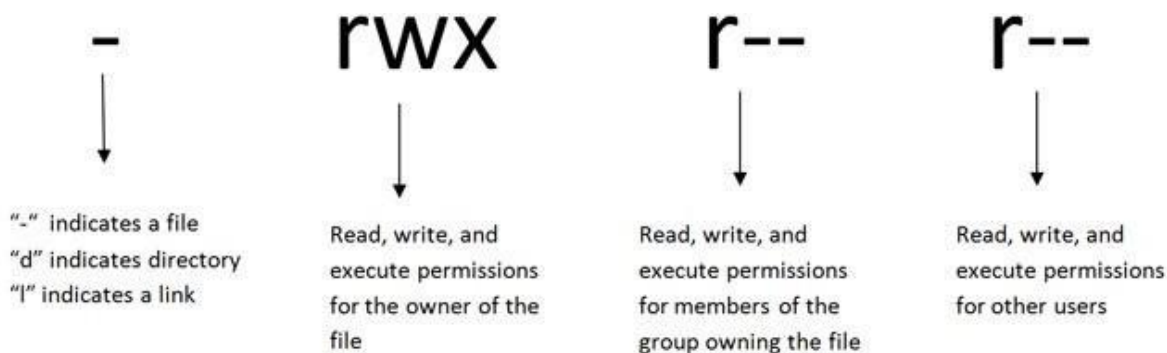
Fourth Column – Represents the group of the owner. Every Unix user will have an associated group.

Fifth Column – Represents the file size in bytes.

Sixth Column – Represents the date and the time when this file was created or modified for the last time.

Seventh Column – Represents the file or the directory name.

4.2 GRANTING PERMISSIONS



To change directory permissions in Linux, use the following: -

1. Chmod + rwx filename to add permissions.
2. Chmod – rwx directoryname to remove permissions
3. Chmod +x filename to allow executable permissions
4. Chmod –wx filename to take out write and executable permissions.

Note that “r” is for read, “w” is for write, and “x” is for execute.

This only changes the permissions for the owner of the file.

We may need to know how to change permissions in numeric code in Linux, so to do this you use numbers instead of “r”, “w”, or “x”.

0 = No permission

1 = Execute

2 = Read

2 = Write

Basically, we add up the numbers depending on the level of permission we want to give.

Permission numbers are:

0 = ---

1 = --x

2 = -w-

3 = -wx

4 = r-

5 = r-x

6 = rw-

7 = rwx

CHAPTER 5

USER MANAGEMENT

User management includes everything from creating a user to deleting a user on your system. User management can be done in three ways on a Linux system.

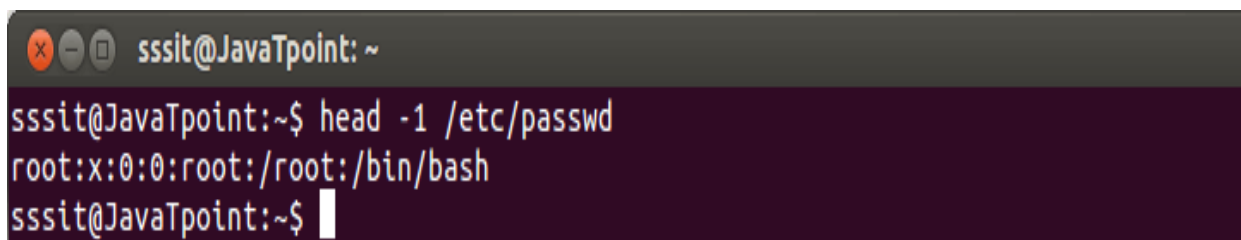
Graphical tools are easy and suitable for new users, as it makes sure you'll not run into any trouble.

Command line tools include commands like `useradd`, `userdel`, `passwd`, etc. These are mostly used by the server administrators.

Third and very rare tool is to edit the local configuration files directly using `vi`.

ROOT USER

The root user is the superuser and has all the powers for creating a user, deleting a user and can even login with the other user's account.



```
sssit@JavaTpoint: ~  
sssit@JavaTpoint:~$ head -1 /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
sssit@JavaTpoint:~$
```

5.1 ADD USER:

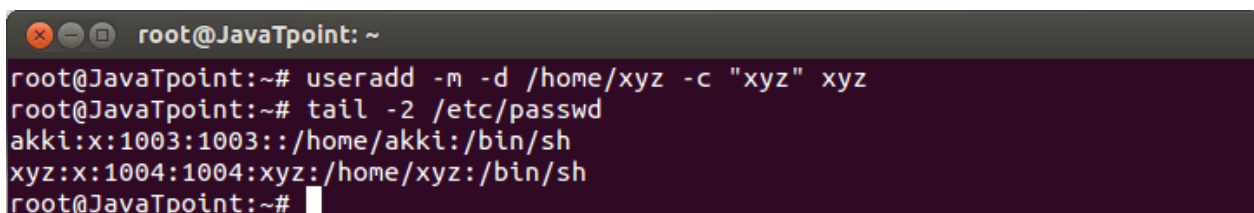
With `useradd` commands you can add a user.

Syntax:

```
useradd -m -d /home/<username> -e "<username>" <userName>
```

Example:

```
useradd -m -d /home/xyz -e "xyz" xyz
```



```
root@JavaTpoint: ~  
root@JavaTpoint:~# useradd -m -d /home/xyz -c "xyz" xyz  
root@JavaTpoint:~# tail -2 /etc/passwd  
akki:x:1003:1003::/home/akki:/bin/sh  
xyz:x:1004:1004:xyz:/home/xyz:/bin/sh  
root@JavaTpoint:~#
```

DELETE USER:

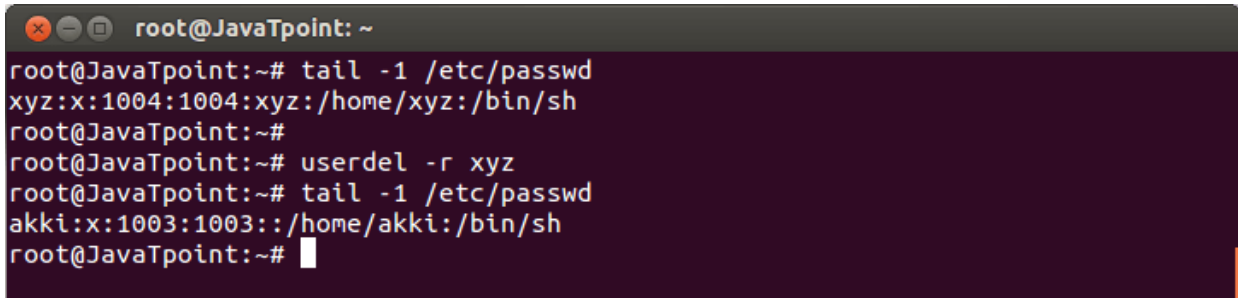
To delete a user account `userdel` command is used.

Syntax:

```
userdel -r <username>
```

Example:

```
userdel -r xyz
```

A terminal window titled 'root@JavaTpoint: ~' showing the process of deleting a user. The user 'xyz' is first identified by checking the last line of /etc/passwd. Then, 'userdel -r xyz' is executed. Finally, the last line of /etc/passwd is checked again, showing that 'xyz' has been removed and 'akki' is now the last entry.

```
root@JavaTpoint:~# tail -1 /etc/passwd
xyz:x:1004:1004:xyz:/home/xyz:/bin/sh
root@JavaTpoint:~#
root@JavaTpoint:~# userdel -r xyz
root@JavaTpoint:~# tail -1 /etc/passwd
akki:x:1003:1003::/home/akki:/bin/sh
root@JavaTpoint:~#
```

5.2 ADDING AND DELETING HOME DIRECTORIES

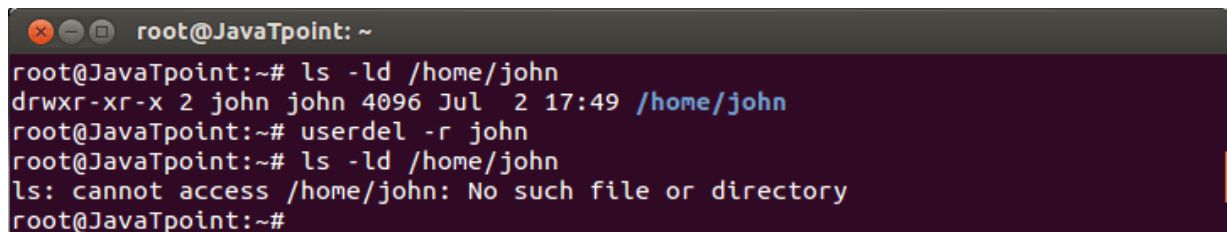
By using `userdel -r` option, you can delete home directory along with user account.

Syntax:

```
Userdel -r <username>
```

Example:

```
Userdel -r john
```

A terminal window titled 'root@JavaTpoint: ~' showing the deletion of a user's home directory. First, 'ls -ld /home/john' is run to confirm the directory exists. Then, 'userdel -r john' is executed. Finally, 'ls -ld /home/john' is run again, resulting in an error message: 'ls: cannot access /home/john: No such file or directory', confirming the directory has been removed.

```
root@JavaTpoint:~# ls -ld /home/john
drwxr-xr-x 2 john john 4096 Jul  2 17:49 /home/john
root@JavaTpoint:~# userdel -r john
root@JavaTpoint:~# ls -ld /home/john
ls: cannot access /home/john: No such file or directory
root@JavaTpoint:~#
```

MODIFY USER:

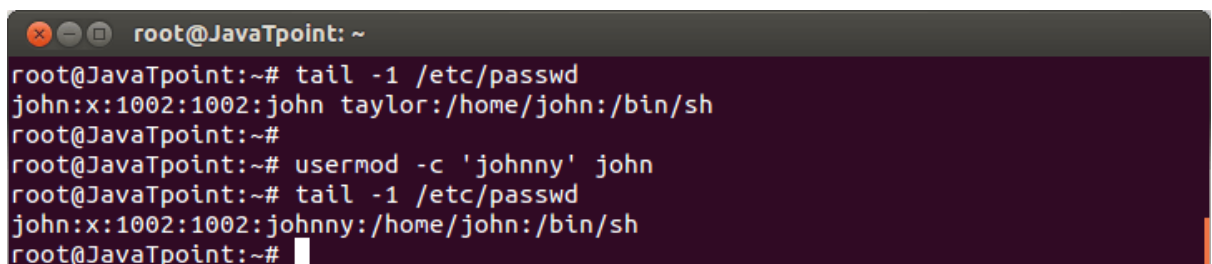
The command `usermod` is used to modify the properties of an existing user.

Syntax:

```
usermod -e <'newName'><oldName>
```

Example:

```
usermod -e 'jhonny' john
```

A terminal window titled 'root@JavaTpoint: ~' showing a user modification. The user 'john' is identified by checking the last line of /etc/passwd. Then, 'usermod -c 'jhonny' john' is executed. Finally, the last line of /etc/passwd is checked again, showing that the comment field for 'john' has been updated to 'jhonny'.

```
root@JavaTpoint:~# tail -1 /etc/passwd
john:x:1002:1002:john taylor:/home/john:/bin/sh
root@JavaTpoint:~#
root@JavaTpoint:~# usermod -c 'jhonny' john
root@JavaTpoint:~# tail -1 /etc/passwd
john:x:1002:1002:johnny:/home/john:/bin/sh
root@JavaTpoint:~#
```

CREATING DIRECTORIES

We will now understand how to create directories. Directories are created by the following command-

```
$mkdirdirname
```

Here, directory is the absolute or relative pathname of the directory you want to create. For example, the command

```
$mkdirmydir  
$
```

Creates the directory mydir in the current directory. Here is another example –

```
$mkdir /tmp/test-dir  
$
```

This command creates the directory test-dir in the/tmp directory.

The mkdir command produces no output if it successfully creates the requested directory.

REMOVING DIRECTORIES

Directories can be deleted using the rmdir command as follows –

```
$rmdirdirname  
$
```

CHAPTER 6

SHELL SCRIPTING

6.1 STEPS TO WRITE AND EXECUTE A SCRIPT

1. Open the terminal. Go to the directory where you want to create your script.
2. Create a file with .sh extension.
3. Write the script in the file using an editor.
4. Make the script executable with command `chmod +x <file- Name>`.
5. Run the script using `./<fileName>`.

A shell script is a computer program designed to be run by the Unix/Linux shell which could be one of the following:

- The Bourne Shell
- The C Shell
- The Korn Shell
- The GNU Bourne-Again Shell

A shell is a command-line interpreter and typical operations performed by shell scripts include file manipulation, program execution, and printing text.

The following script uses the `read` command which takes the input from the keyboard and assigns it as the value of the variable `PERSON` and finally prints it on `STDOUT`.

```
#!/bin/sh

# Author : Zara Ali
# Copyright (c) Tutorialspoint.com#
Script follows here:

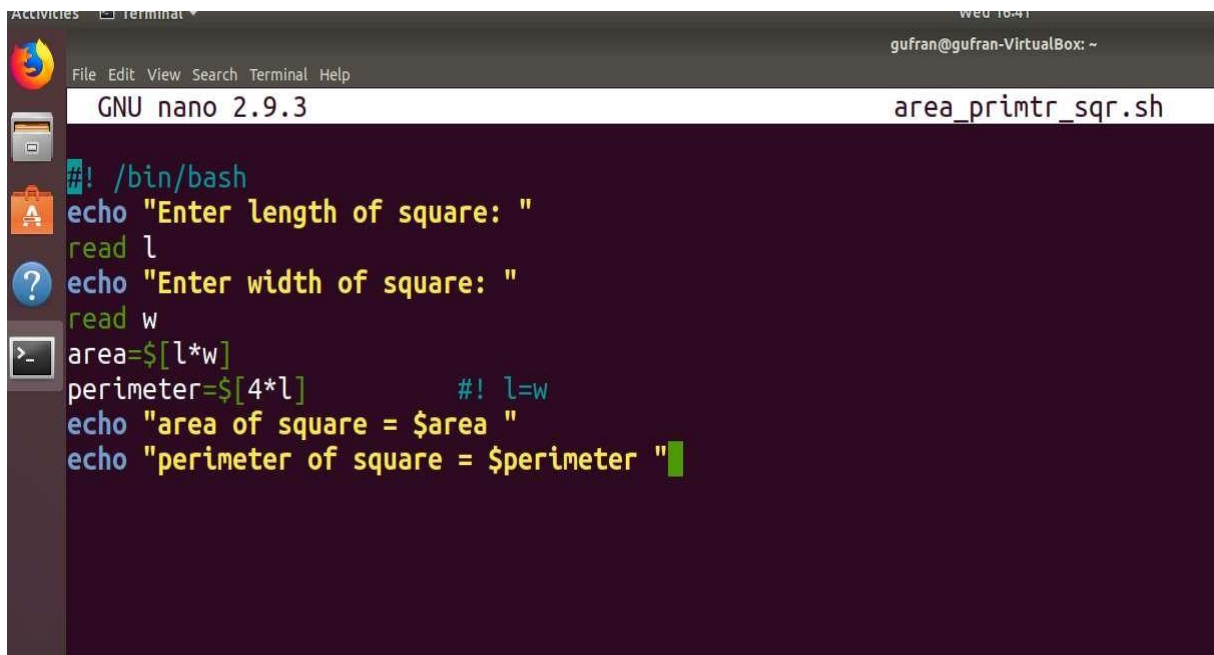
echo "What is your name?"read
PERSON
echo "Hello, $PERSON"
```

Here is a sample run of the script –

```
./test.sh
What is your name?Zara Ali
Hello, Zara Ali
$
```

6.2 LIST OF PROGRAM OF SHELLSCRIPTING

Q1 Write a shell script to enter length & width of square and calculate its area & perimeter and display them.

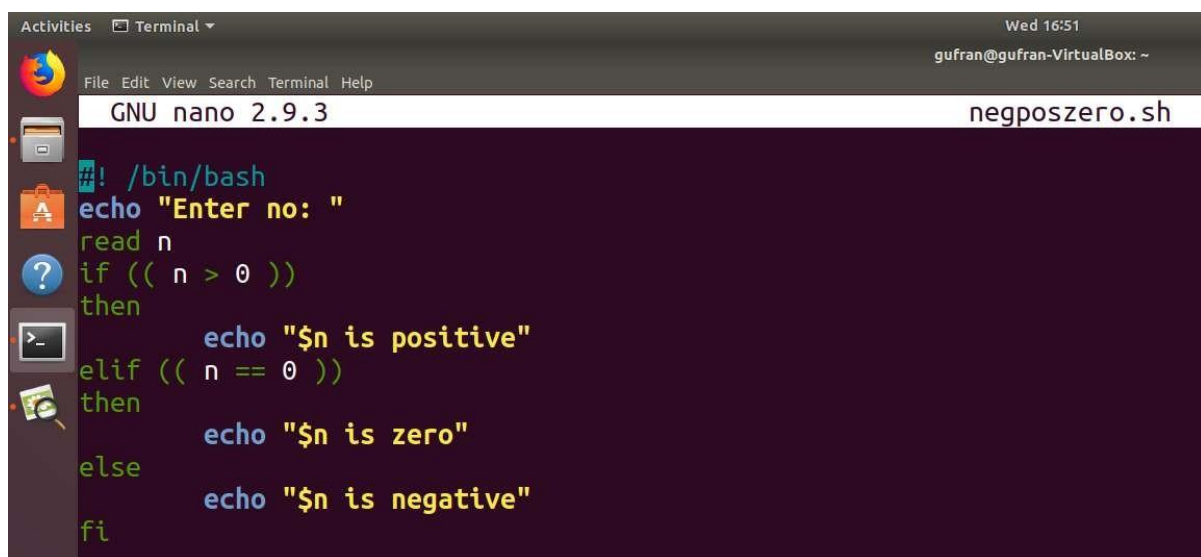


The screenshot shows a terminal window with the GNU nano 2.9.3 editor open. The file being edited is named 'area_printr_sqr.sh'. The script content is as follows:

```
#!/bin/bash
echo "Enter length of square: "
read l
echo "Enter width of square: "
read w
area=$((l*w))
perimeter=$((4*l))      #! l=w
echo "area of square = $area "
echo "perimeter of square = $perimeter "
```

```
gufran@gufran-VirtualBox:~$ ./area_primtr_sqr.sh
Enter length of square:
8
Enter width of square:
8
area of square = 64
perimeter of square = 32
```

Q2 write a shell script to check whether a number is positive, negative or zero.



```
Activities  Terminal  Wed 16:51
gufran@gufran-VirtualBox: ~
GNU nano 2.9.3 negposzero.sh
#!/bin/bash
echo "Enter no: "
read n
if (( n > 0 ))
then
    echo "$n is positive"
elif (( n == 0 ))
then
    echo "$n is zero"
else
    echo "$n is negative"
fi
```

```
gufran@gufran-VirtualBox:~$ ./negposzero.sh
Enter no:
4
4 is positive
gufran@gufran-VirtualBox:~$ ./negposzero.sh
Enter no:
-5
-5 is negative
gufran@gufran-VirtualBox:~$ ./negposzero.sh
Enter no:
0
0 is zero
```

Q3 write a shell script to print all natural numbers in recursive (from n to 1). – using while loop.

```
echo "enter the no"
read n
while [ $n != 0 ]
do
echo "$n"
n=$(( n - 1 ))
done
```

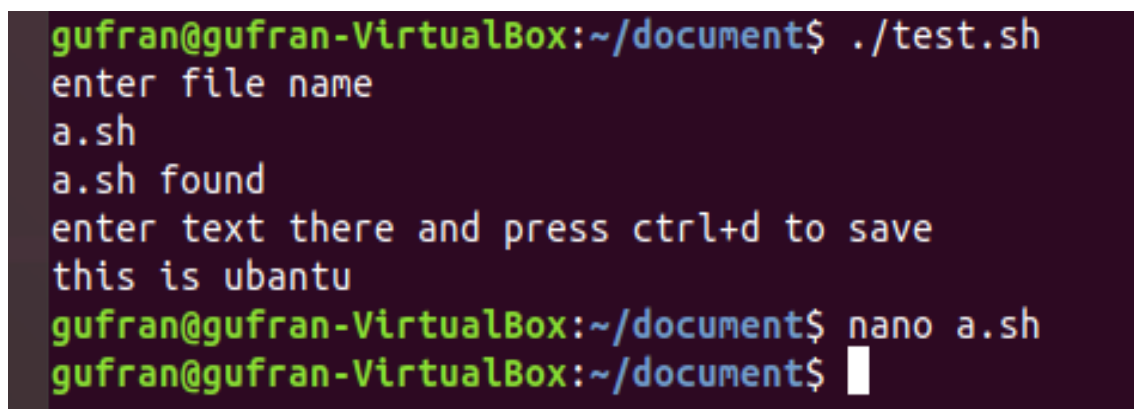
```
uttam@uttam-VirtualBox:~$ .
enter the no
5
5
4
3
2
1
uttam@uttam-VirtualBox:~$
```

Q5 Write a shell script to input file name and check if file found ask user to enter some text and save it into file, if not found display message not found.

A screenshot of a nano editor window titled 'GNU nano 2.9.3'. The script content is as follows:

```
#!/bin/bash
echo "enter file name"
read file

if [ -f $file ]
then
    echo "$file found"
    echo "enter text there and press ctrl+d to save"
    cat >> $file
else
    echo "$file not found"
fi
```

A screenshot of a terminal window showing the execution of the script. The prompt is 'gufran@gufran-VirtualBox:~/document\$'. The user enters './test.sh', and the script outputs 'enter file name', 'a.sh', and 'a.sh found'. It then prompts 'enter text there and press ctrl+d to save', where the user enters 'this is ubuntu'. Finally, the user enters 'nano a.sh' and the prompt returns to 'gufran@gufran-VirtualBox:~/document\$'.

CHAPTER 7

CISCO PACKET TRACER

INTRODUCTION

Cisco Packet Tracer is a network simulation tool that allows users to create and simulate network topologies using virtual devices and connections. It is a popular tool for network engineers, students, and instructors to design, build, and test network configurations without the need for physical hardware.

Creating a Cisco Packet Tracer project typically involves designing a network topology and configuring various devices within that topology to achieve specific objectives. Here's a general outline of how you might approach creating a project:

Define Requirements: Clearly define the objectives and requirements of your network project. For example, it could be to set up a small office network, a campus network, or simulate a specific scenario like network security or VoIP implementation.

Design Topology: Use Cisco Packet Tracer's interface to design your network topology. Drag and drop routers, switches, PCs, servers, and other devices onto the workspace. Connect these devices using appropriate cables (Ethernet, serial, etc.).

Configure Devices: Access each device (router, switch, PC, etc.) by double-clicking on it and configure its settings. This includes assigning IP addresses, setting up routing protocols (if necessary), configuring VLANs, setting up DHCP servers, etc.

Implement Services: Depending on your project requirements, implement various network services such as DHCP, DNS, NAT, ACLs (Access Control Lists), VPNs, and others.

Test and Verify: Once configured, simulate the network to ensure everything is functioning as expected. Test connectivity between devices, verify that services are working correctly, and troubleshoot any issues that arise.

Documentation: Document your project thoroughly. Include details of your network design, configurations, and any specific settings or features implemented.

Presentation (if required): If your project requires a presentation, prepare slides or a report summarizing your design choices, configurations, and the overall functionality of your network.

Cisco Packet Tracer provides a user-friendly environment for these tasks, making it suitable for both beginners and advanced users to explore and implement networking concepts.

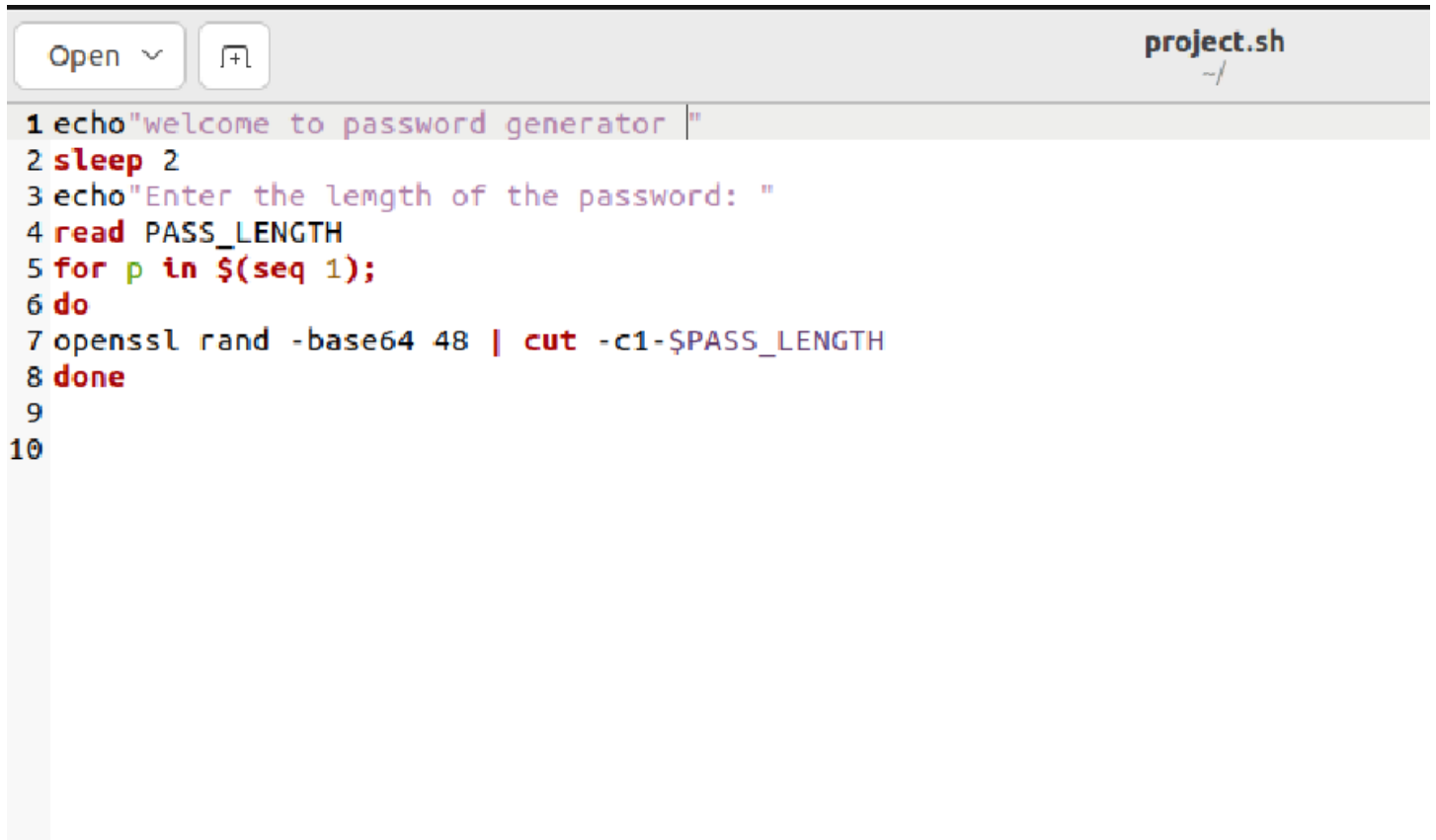
Key Features:

- Unlimited devices
- E-learning
- Customize single/multi user activities
- Interactive Environment
- Visualizing Networks
- Real-time mode and Simulation mode
- Self-paced
- Supports majority of networking protocols
- International language support
- Cross platform compatibility

CHAPTER 8

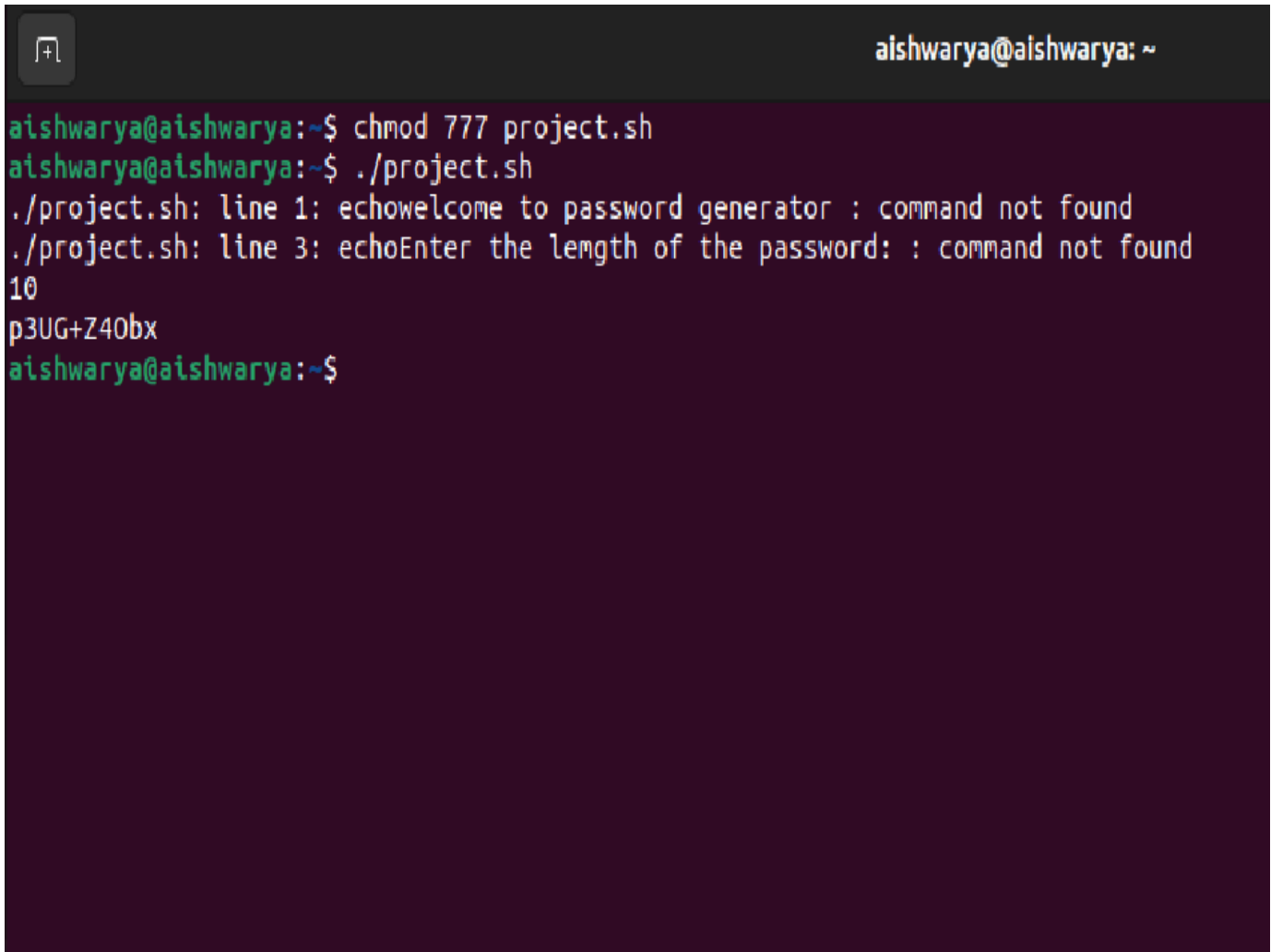
OUTCOMES

SNAPSHOTS



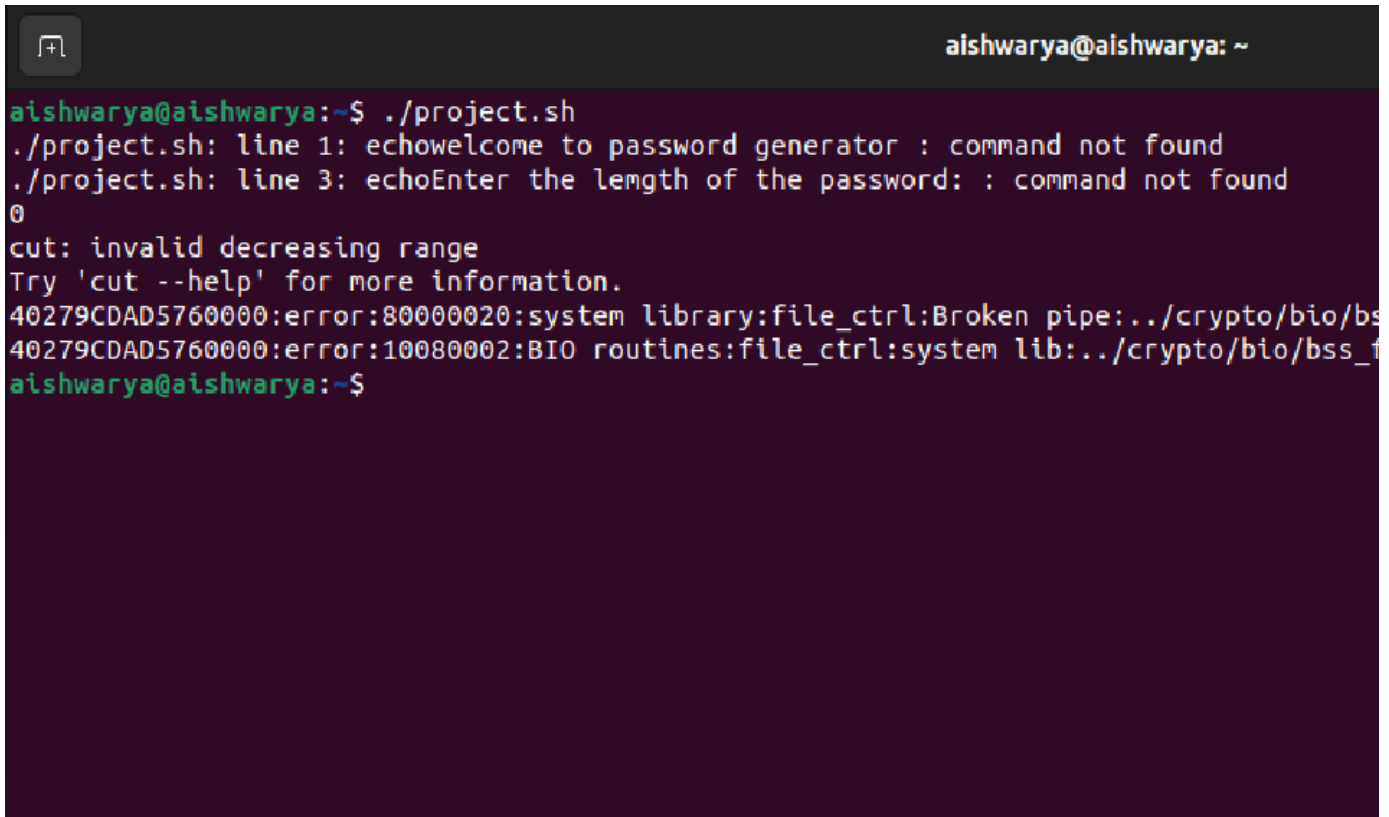
```
1 echo "welcome to password generator "
2 sleep 2
3 echo "Enter the length of the password: "
4 read PASS_LENGTH
5 for p in $(seq 1);
6 do
7 openssl rand -base64 48 | cut -c1-$PASS_LENGTH
8 done
9
10
```

FIG 8.1 : PASSWORD GENARATOR CODE

A terminal window with a dark purple background. The title bar at the top shows a window icon on the left and the text 'aishwarya@aishwarya: ~' on the right. The terminal content shows a user at the 'aishwarya@aishwarya:~\$' prompt. They enter 'chmod 777 project.sh'. At the next prompt, they enter './project.sh'. The script outputs two error messages: './project.sh: line 1: echowelcome to password generator : command not found' and './project.sh: line 3: echoEnter the length of the password: : command not found'. Then, the number '10' is entered, followed by the generated password 'p3UG+Z40bx'. The terminal returns to the 'aishwarya@aishwarya:~\$' prompt.

```
aishwarya@aishwarya:~$ chmod 777 project.sh
aishwarya@aishwarya:~$ ./project.sh
./project.sh: line 1: echowelcome to password generator : command not found
./project.sh: line 3: echoEnter the length of the password: : command not found
10
p3UG+Z40bx
aishwarya@aishwarya:~$
```

FIG 8.2: RANDOM PASSWORD GENARATED

A terminal window with a dark purple background. The title bar at the top right says 'aishwarya@aishwarya: ~'. The terminal shows the execution of a script named 'project.sh'. The first line of the script is 'echo welcome to password generator : command not found'. The second line is 'echo Enter the length of the password: : command not found'. The third line is '0'. The fourth line is 'cut: invalid decreasing range'. The fifth line is 'Try 'cut --help' for more information.'. The sixth line is '40279CDAD5760000:error:80000020:system library:file_ctrl:Broken pipe:../crypto/bio/bss_1'. The seventh line is '40279CDAD5760000:error:10080002: BIO routines:file_ctrl:system lib:../crypto/bio/bss_1'. The eighth line is 'aishwarya@aishwarya:~\$'.

```
aishwarya@aishwarya:~$ ./project.sh
./project.sh: line 1: echo welcome to password generator : command not found
./project.sh: line 3: echo Enter the length of the password: : command not found
0
cut: invalid decreasing range
Try 'cut --help' for more information.
40279CDAD5760000:error:80000020:system library:file_ctrl:Broken pipe:../crypto/bio/bss_1
40279CDAD5760000:error:10080002: BIO routines:file_ctrl:system lib:../crypto/bio/bss_1
aishwarya@aishwarya:~$
```

FIG 8.3 : INVALID PASSWORD

REFERENCES

- 1) https://linuxcommand.org/lc3_learning_the_shell.php
- 2) https://linuxcommand.org/lc3_resources.php
- 3) <https://www.geeksforgeeks.org/introduction-to-linux-operating-system/>
- 4) <https://www.geeksforgeeks.org/linux-commands/?ref=lbp>
- 5) <https://www.scribd.com/document/647840691/OPS>
- 6) <https://www.ijtrd.com/papers/IJTRD1374.pdf>

CONCLUSION

Linux is a Unix-like, open source and community-developed operating system (OS) for computers, servers, mainframes, mobile devices and embedded devices. It is supported on almost every major computer platform, including x86, ARM and SPARC, making it one of the most widely supported operating systems. Linux is not just an operating system; it's a symbol of the collaborative power of the open-source community and a testament to the importance of free and customizable software.

an operating system is a fundamental component of a computer system. It performs various functions, such as process and memory management, file system management, and device management.